

Template Week 4 – Software

Student number: 587890

Assignment 4.1: ARM assembly

Screenshot of working assembly code of factorial calculation:

main:

```
mov    r1, #5    @ Number to calculate factorial of (n)
mov    r0, #1    @ Result = 1
```

Loop:

```
cmp    r1, #1    @ while (n > 1)
ble    End       @ if n <= 1, exit loop

mul    r0, r0, r1 @ result *= n
sub    r1, r1, #1 @ n--
b      Loop
```

End:

```
bx     lr        @ return, result in r0
```

Assignment 4.2: Programming languages

Take screenshots that the following commands work:

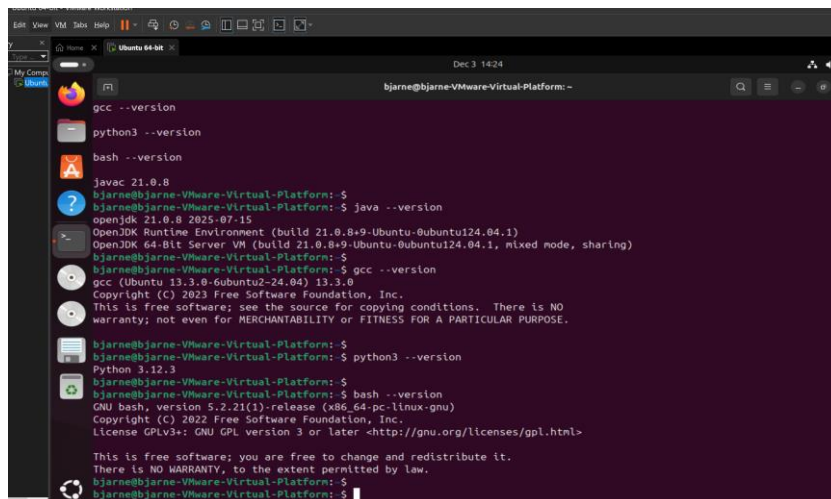
```
javac --version
```

```
java --version
```

```
gcc --version
```

```
python3 --version
```

```
bash --version
```



```
gcc --version
python3 --version
bash --version
javac 21.0.8
bjarne@bjarne-VMware-Virtual-Platform: $ java --version
bjarne@bjarne-VMware-Virtual-Platform: $ gcc --version
OpenJDK 21.0.8 2025-07-15
OpenJDK Runtime Environment (build 21.0.8+9-Ubuntu-0ubuntu124.04.1)
OpenJDK 64-Bit Server VM (build 21.0.8+9-Ubuntu-0ubuntu124.04.1, mixed mode, sharing)
bjarne@bjarne-VMware-Virtual-Platform: $ python3 --version
Python 3.12.3
bjarne@bjarne-VMware-Virtual-Platform: $ bash --version
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
bjarne@bjarne-VMware-Virtual-Platform: $
```

Assignment 4.3: Compile

Which of the above files need to be compiled before you can run them? The c file and the java file

Which source code files are compiled into machine code and then directly executable by a processor?
The c file

Which source code files are compiled to byte code? the java file

Which source code files are interpreted by an interpreter? python

These source code files will perform the same calculation after compilation/interpretation. Which one is expected to do the calculation the fastest? the c file

How do I run a Java program? `javac programname.java` and that java programname

How do I run a Python program? `python programname.py`

How do I run a C program? `Gcc programname.c -o programname`

How do I run a Bash script? `bash scriptmane.sh`

If I compile the above source code, will a new file be created? If so, which file?

Java makes a .class file

Python no file is made

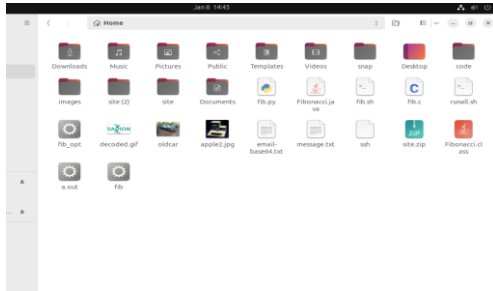
C makes an executable

Bash no file is made

Take relevant screenshots of the following commands:

- Compile the source files where necessary
- Make them executable
- Run them
- Which (compiled) source code file performs the calculation the fastest?

```
second$  
  
tual-Platform:~$ gcc fib.c  
tual-Platform:~$ javac fib.java  
fib.java  
source files>  
possible options  
tual-Platform:~$ javac Fibonacci.java  
tual-Platform:~$ S
```



```
bjarne@bjarne:~$ gcc -O0 fib.c -o fib_00
Running C program:
Fibonacci(19) = 4181
Execution time: 0.03 milliseconds

bjarne@bjarne:~$ gcc -O1 fib.c -o fib_01
Running Java program:
Fibonacci(19) = 4181
Execution time: 0.28 milliseconds

bjarne@bjarne:~$ gcc -O2 fib.c -o fib_02
Running Python program:
Fibonacci(19) = 4181
Execution time: 0.63 milliseconds

bjarne@bjarne:~$ gcc -O3 fib.c -o fib_03
Running BASH Script
S
```

Assignment 4.4: Optimize

Take relevant screenshots of the following commands:

- Figure out which parameters you need to pass to **the gcc** compiler so that the compiler performs a number of optimizations that will ensure that the compiled source code will run faster. **Tip!** The parameters are usually a letter followed by a number. Also read **page 191** of your book, but find a better optimization in the man pages. Please note that Linux is case sensitive.
 - O0 – No optimization (standard)
 - O1 – Basic optimization
 - O2 – More aggressive optimizations (recommended general use)
 - O3 – Even more aggressive (loop unrolling, vectorization, etc.)
 - Ofast – Enables -O3 plus non-standard optimizations (But could break strict standards compliance)
- Compile **fib.c** again with the optimization parameters

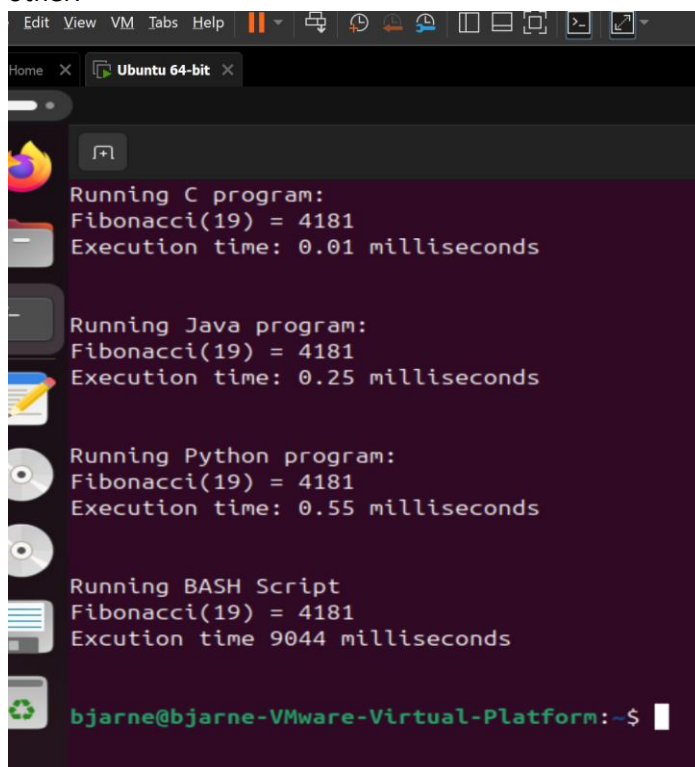
```
~$ gcc -O3 fib.c -o fib_03
~$ gcc -O3 fib.c -o fib_03
~$ ./fib_00
```

- c) Run th

```
/fib_00: No such file or directory
e-VMware-Virtual-Platform:~$ ./fib_03
) = 2584
me: 0.01 milliseconds
e-VMware-Virtual-Platform:~$ ./fib
) = 2584
me: 0.02 milliseconds
e-VMware-Virtual-Platform:~$
```

e newly compiled program. Is it true that it now performs the calculation faster?

- d) Edit the file `runall.sh`, so you can perform all four calculations in a row using this Bash script. So the (compiled/interpreted) C, Java, Python and Bash versions of Fibonacci one after the other.



```
Edit View VM Tabs Help
Home x Ubuntu 64-bit x
Running C program:
Fibonacci(19) = 4181
Execution time: 0.01 milliseconds

Running Java program:
Fibonacci(19) = 4181
Execution time: 0.25 milliseconds

Running Python program:
Fibonacci(19) = 4181
Execution time: 0.55 milliseconds

Running BASH Script
Fibonacci(19) = 4181
Execution time 9044 milliseconds

bjarne@bjarne-VMware-Virtual-Platform:~$
```

Assignment 4.5: More ARM Assembly

Like the factorial example, you can also implement the calculation of a power of 2 in assembly. For example you want to calculate $2^4 = 16$. Use iteration to calculate the result. Store the result in r0.

Main:

```
mov    r1, #2
mov    r2, #4
mov    r0, #1
```

Loop:

```
cmp    r2, #0
beq    End

mul    r0, r0, r1
sub    r2, r2, #
b      Loop
```

End:

```
bx     lr
```

Complete the code. See the PowerPoint slides of week 4.

Screenshot of the completed code here.

Main:

```
mov    r1, #2
mov    r2, #4
mov    r0, #1
```

Loop:

```
cmp    r2, #0
beq    End

mul    r0, r0, r1
sub    r2, r2, #1
b      Loop
```

End:

```
bx     lr
```

Ready? Save this file and export it as a pdf file with the name: [week4.pdf](#)